
Proper Time for Verified Model Selection

Anonymous Authors

Abstract

1 Introduction

Achille and Soatto argue that agents should be understood as task solvers whose performance is measured by *time to solution*: the resource required to produce a verified artifact, not the model’s self-assessment of progress [Achille and Soatto, 2026]. This perspective changes a basic practical question. Modern model choice is usually treated as a leaderboard problem: run each candidate on a benchmark and pick the one with the highest average score. For verified tasks, this is often the wrong unit of comparison. In code generation, repository repair, formal reasoning, and exact-answer reasoning benchmarks, a system is useful only when it produces an artifact accepted by an external verifier. The deployment question is therefore not merely whether a model can eventually solve an instance, but how much wall-clock time, token budget, and sampling effort must be spent before a verified solution appears.

This distinction matters because verified performance is trajectory-valued. A small model may be cheap enough that several retries are preferable to one call to a larger model. A larger model may be slower but decisive on modes where smaller models repeatedly fail. A router may observe enough information about a fresh instance to select the right worker, or it may waste that information by choosing an action that is not on the mode-conditioned frontier. Average accuracy collapses these mechanisms into one number, even though they imply different engineering decisions.

The scientific reason to introduce task modes is the same reason algorithm selection and portfolio methods exist: instance structure can change which solver is cost-effective. In Rice’s algorithm-selection formulation, one chooses an algorithm using observable features of the problem instance [Rice, 1976]; SATzilla and AutoFolio made this idea practical by learning when different solvers dominate on different regions of a task distribution [Xu et al., 2008, Lindauer et al., 2015]. In machine learning, transferability and task-embedding work similarly treats tasks as structured objects rather than interchangeable samples [Achille et al., 2019]. Our modes play this role for verified language model deployment. They are not post-hoc names for success or failure. They are predeclared, solver-relevant partitions or covariates—for example difficulty strata, algorithmic categories, or reasoning families—under which different models may have different cost–success curves.

This makes a concrete prediction. If one model has the best cost-adjusted first-passage profile in every mode, routing is trivial: no mode label can make a different model worth running. If the frontier crosses across modes, then model choice becomes conditional. A smaller or cheaper model can be optimal under a wall-clock, token, or price budget on modes where its success rate is high enough; a larger model can be optimal on modes where the smaller model’s retry curve saturates below the required reliability. This is the verified model-selection analogue of comparative advantage.

We do not introduce proper time as a new philosophical object. Instead, we make the Achille–Soatto principle empirical. Given a fixed verifier, a resource clock, and repeated stochastic runs, we estimate the first-passage curve

$$F_M(t) = \Pr[\text{model } M \text{ has produced a verified solution by resource } t],$$

* Equal contribution. Correspondence to {pierb, erimoldi}@mit.edu.

keeping unsolved instances as right-censored observations. From this curve we study the operating frontier

$$\tau_M^* = \inf_t \frac{t}{F_M(t)},$$

which asks how much resource must be paid per unit of verified success at the best operating point. This turns model selection into verified first-passage accounting rather than benchmark racing.

The main observation is that proper-time differences are not monolithic. A system can improve because its attempts are cheaper, because its worker has higher verified competence on a task mode, because the instance exposes information about which mode it belongs to, or because a router allocates that information well. We therefore organize verified model selection around four diagnostic quantities:

1. **Cost:** the resource consumed by an attempt or by a stop-at-first-success trajectory.
2. **Competence:** the mode-conditioned first-passage success curve of a worker under a fixed verifier.
3. **Information:** the value of knowing a mode or router-visible signal before deployment.
4. **Allocation regret:** the loss of the implemented router relative to the best action available under the same information.

The resulting decomposition separates two questions that are often conflated: whether there is useful specialization in the model menu, and whether a router actually exploits it.

We instantiate this protocol on verified coding and reasoning tasks. For each worker model we log every sampling attempt, the per-attempt resource usage, the verifier outcome, and the cumulative resource to the first passing attempt; runs that never pass remain in the data as censored observations. This trace format lets us evaluate single-model frontiers, retry policies, oracle allocations, statistical oracles that learn from held-out folds, and prompt-only LLM routers without rerunning workers. The router never writes a solution; it only chooses an allocation over already logged worker attempts, and is judged by the verified time-to-solution that allocation would have incurred.

Our contributions are:

1. We operationalize proper time for verified model selection as an estimable first-passage object with wall-clock and token clocks.
2. We give a diagnostic decomposition of proper-time gains into cost, competence, information value, and allocation regret.
3. We test the retry-crossover prediction empirically, distinguishing the closed-form Bernoulli approximation from the heterogeneous first-passage curves observed in real task sets.
4. We evaluate when routing is non-trivial: a router can beat the best single model only when task modes induce cost-adjusted frontier crossings, and only if the router uses the available information well.

The paper’s central message is simple: accuracy tells us whether a model can solve; proper time tells us whether it is worth running.

2 Verified Tasks and First Passage

We begin with the object on which proper time is measured. The task must have an external success condition: the model may propose artifacts, but the acceptance event is determined by a verifier outside the model.

Definition 1 (Verified task). *A verified task is a tuple*

$$\mathcal{T} = (\mathcal{X}, \mathcal{Y}, \mu, V),$$

where $\mathcal{X}$ is an instance space, $\mathcal{Y}$ is an artifact space, μ is a distribution over instances, and

$$V : \mathcal{X} \times \mathcal{Y} \rightarrow \{0, 1\}$$

is an external verifier. For an instance $x \in \mathcal{X}$ and candidate artifact $y \in \mathcal{Y}$, the event $V(x, y) = 1$ means that y is accepted as a verified solution to x .

The definition is intentionally agnostic about the kind of artifact. In a code task, x is a programming problem and y is a generated program; the verifier runs the declared tests. In an exact-answer reasoning task, x is a question and y is the model’s final answer; the verifier applies a frozen extraction and normalization rule and checks exact match. Scalar evaluators can be included by thresholding: if $R(x, y) \in \mathbb{R}$ is an external reward or loss, then $V_\gamma(x, y) = \mathbf{1}\{R(x, y) \geq \gamma\}$ for a predeclared threshold γ .

Two points are important. First, verification is external: success is not the model’s claim that the answer is correct. Second, the verifier defines the unit of progress. Any comparison of models, retries, or routers in this paper is made relative to the same task distribution and the same frozen verifier.

Definition 2 (Deployment action and resource clock). *Let $\mathcal{A}$ be a finite set of deployment actions, and fix a deployment budget $B \in (0, \infty]$. An action $a \in \mathcal{A}$ is any fixed procedure that, given an instance x and internal randomness $r \in \Omega$, emits candidate artifacts before the budget is exhausted. Its execution is the finite or countable sequence*

$$\Gamma_B^a(x, r) = (C_k^a(x, r), Y_k^a(x, r))_{k=1}^{K_B^a(x, r)}, \quad K_B^a(x, r) \in \mathbb{N}_0 \cup \{\infty\},$$

where $K_B^a(x, r)$ is the number of candidate artifacts emitted within budget, $Y_k^a(x, r) \in \mathcal{Y}$ is the k -th candidate artifact, and $C_k^a(x, r)$ is the cumulative resource spent when that artifact becomes available. When $K_B^a(x, r) \geq 1$, the clock is cumulative:

$$0 < C_1^a(x, r) \leq C_2^a(x, r) \leq \dots, \quad C_k^a(x, r) \leq B \quad \text{for } 1 \leq k \leq K_B^a(x, r).$$

The resource may be wall-clock seconds, generated tokens, dollar cost, verifier calls, or a fixed weighted combination of these quantities. Examples include a single language-model call, a language model wrapped in a harness, a repeated-sampling policy, a sequential allocation across models, a multi-agent system, or a broader orchestration capable of solving the instance.

The cumulative convention is part of the definition. If a repeated-sampling action succeeds on its third candidate, the cost of the first two failed candidates is still paid. Likewise, if a sequential allocation tries a cheap worker before an expensive one, the verified time of the allocation includes every consumed attempt up to and including the first accepted artifact.

Definition 3 (Verified first-passage time). *For a verified task $\mathcal{T}$, action a , budget B , instance x , and randomness r , the first verified index is*

$$J_B^a(x, r) := \min \{k \in \mathbb{N} : k \leq K_B^a(x, r), V(x, Y_k^a(x, r)) = 1\},$$

with $J_B^a(x, r) = \infty$ when the set is empty. The verified first-passage time is

$$\tau_B^a(x, r) := \begin{cases} C_{J_B^a(x, r)}^a(x, r), & J_B^a(x, r) < \infty, \\ \infty, & J_B^a(x, r) = \infty. \end{cases}$$

Thus $\tau_B^a(x, r)$ is the cumulative resource spent up to the first accepted artifact. If no artifact is accepted within budget, the run is right-censored at B and represented by $\tau_B^a = \infty$ for any finite resource threshold.

Definition 4 (Certified time-to-solution). *For confidence level $\delta \in (0, 1)$, the certified time-to-solution of action a is the $(1 - \delta)$ -success quantile of its verified first-passage time:*

$$\bar{T}_a(\delta) := \inf \left\{ t > 0 : \Pr_{X \sim \mu, r} [\tau_B^a(X, r) \leq t] \geq 1 - \delta \right\}.$$

Equivalently, if

$$F_a(t) := \Pr_{X \sim \mu, r} [\tau_B^a(X, r) \leq t],$$

then $\bar{T}_a(\delta) = \inf\{t > 0 : F_a(t) \geq 1 - \delta\}$. This is the quantile form of proper time. It is infinite whenever the action’s success probability within budget is below $1 - \delta$.

Definition 5 (Success-by-budget curve). *The success-by-budget curve of action a is*

$$F_a(t) := \Pr_{X \sim \mu, r} [\tau_B^a(X, r) \leq t], \quad 0 \leq t \leq B.$$

Given observed runs $\{(x_i, r_i)\}_{i=1}^n$, its empirical estimate is

$$\hat{F}_a(t) := \frac{1}{n} \sum_{i=1}^n \mathbf{1}\{\tau_B^a(x_i, r_i) \leq t\}.$$

Runs with no verified solution have $\tau_B^a = \infty$; they remain in the denominator and contribute zero to the numerator for every finite t .

Thus benchmark accuracy at budget B is the single number $F_a(B)$, while proper-time accounting uses the full first-passage curve $F_a(t)$.

Definition 6 (Modes and mode-conditioned proper time). *A mode schema is a finite set $\mathcal{S}$ together with a predeclared map*

$$\sigma : \mathcal{X} \rightarrow \mathcal{S}.$$

We write $S = \sigma(X)$ and $\pi_s = \Pr[S = s]$. For any mode with $\pi_s > 0$, the mode-conditioned success-by-budget curve is

$$F_{a,s}(t) := \Pr[\tau_B^a(X, r) \leq t \mid S = s].$$

The mode-conditioned operating proper time is

$$\tau_{a,s}^* := \inf_{t: F_{a,s}(t) > 0} \frac{t}{F_{a,s}(t)}.$$

When the infimum is attained, let $t_{a,s}^$ be an attaining resource and define $p_{a,s}^* := F_{a,s}(t_{a,s}^*)$. The log proper-time loss is*

$$\ell(a, s) := \log \tau_{a,s}^* = \underbrace{\log t_{a,s}^*}_{\text{cost}} - \underbrace{\log p_{a,s}^*}_{\text{competence}}.$$

Modes are not defined by whether a particular model succeeds. They are predeclared task structure—difficulty strata, algorithmic categories, reasoning families, or other solver-relevant covariates—used to test whether different actions have different cost-success frontiers on different parts of the task distribution.

Definition 7 (Router and signal). *A router observes a pre-deployment signal $Z = Z(X)$ and selects an action*

$$\rho : \mathcal{Z} \rightarrow \mathcal{A}.$$

Its proper-time loss is

$$L_\rho := \mathbb{E}[\ell(\rho(Z), S)].$$

The best single-action baseline is

$$L_{\text{single}} := \min_{a \in \mathcal{A}} \mathbb{E}[\ell(a, S)].$$

The best decision rule that can use signal Z has loss

$$L_Z^* := \mathbb{E}_Z \left[\min_{a \in \mathcal{A}} \mathbb{E}[\ell(a, S) \mid Z] \right].$$

The value of information and router regret are

$$\text{VOI}(Z) := L_{\text{single}} - L_Z^*, \quad \text{Reg}(\rho; Z) := L_\rho - L_Z^*.$$

Theorem 1 (Operational four-term decomposition). *Assume the mode-conditioned operating points in Definition 6 exist and have finite positive $t_{a,s}^*$ and $p_{a,s}^*$. Fix a reference action a_0 , and let*

$$a_{\text{single}}^* \in \arg \min_{a \in \mathcal{A}} \mathbb{E}[\ell(a, S)]$$

be a best single action. Then any router ρ satisfies

$$\begin{aligned} & \mathbb{E}[\ell(a_0, S)] - L_\rho \\ &= \underbrace{\mathbb{E} \left[\log \frac{t_{a_0, S}^*}{t_{a_{\text{single}}^*, S}^*} \right]}_{\text{cost}} + \underbrace{\mathbb{E} \left[\log \frac{p_{a_{\text{single}}^*, S}^*}{p_{a_0, S}^*} \right]}_{\text{competence}} + \underbrace{\text{VOI}(Z)}_{\text{information}} - \underbrace{\text{Reg}(\rho; Z)}_{\text{allocation regret}}. \end{aligned}$$

Proof. By Definition 6,

$$\ell(a, s) = \log t_{a,s}^* - \log p_{a,s}^*.$$

Therefore

$$\mathbb{E}[\ell(a_0, S)] - L_{\text{single}} = \mathbb{E} \left[\log \frac{t_{a_0, S}^*}{t_{a_{\text{single}}, S}^*} \right] + \mathbb{E} \left[\log \frac{p_{a_{\text{single}}, S}^*}{p_{a_0, S}^*} \right].$$

Adding and subtracting L_{single} and L_Z^* gives

$$\mathbb{E}[\ell(a_0, S)] - L_\rho = (\mathbb{E}[\ell(a_0, S)] - L_{\text{single}}) + (L_{\text{single}} - L_Z^*) - (L_\rho - L_Z^*),$$

which is exactly the stated decomposition. $\square$

The packed latent-mode identity used later can be viewed as a stronger time-sharing surrogate for the same accounting principle: it replaces the action-level regret term with a KL mismatch between a posterior mode distribution and an allocated share vector. The operational decomposition above is the one used for hard action selection, where routers output actions or allocations rather than probability shares over modes.

References

- Alessandro Achille and Stefano Soatto. AI agents as universal task solvers: It’s all about time. *Entropy*, 28(3):332, 2026.
- John R. Rice. The algorithm selection problem. In *Advances in Computers*, volume 15, pages 65–118. Elsevier, 1976.
- Lin Xu, Frank Hutter, Holger H. Hoos, and Kevin Leyton-Brown. SATzilla: Portfolio-based algorithm selection for SAT. *Journal of Artificial Intelligence Research*, 32:565–606, 2008.
- Marius Lindauer, Katharina Eggersperger, Matthias Feurer, Bernd Bischl, and Frank Hutter. AutoFolio: An automatically configured algorithm selector. *Journal of Artificial Intelligence Research*, 53: 745–778, 2015.
- Alessandro Achille, Michael Lam, Rahul Tewari, Avinash Ravichandran, Subhansu Maji, Charless Fowlkes, Stefano Soatto, and Pietro Perona. Task2Vec: Task embedding for meta-learning. In *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, 2019.